

Время: 00:00–04:00

ЭКРАН: доска: агентство → диспетчерская → три Kubernetes-ноды.

ГОВОРЮ

«DEVOPS MAY CRY — наша учебная ночная диспетчерская. В первой части мы собрали для неё кластер и запустили Go-сервис. Теперь решим, где в этом кластере должны жить разные компоненты.

У приложения есть stateless-диспетчеры и PostgreSQL. Диспетчеры можно размножать и раскладывать по workers. Базу иногда приходится привязать к отдельному классу нод: другой диск, больше памяти, отдельная зона отказа. Kubernetes сам не угадает такое решение. Мы задаём его через taints, tolerations, affinity и topology spread.

В этой части я специально получу несколько Pending. Это не поломка стенда. Каждый раз сначала читаю Events, потом меняю ровно одно условие. Так видно, какое правило реально принял Scheduler».

ПОКАЗАТЬ

- `kubernetes/devops-may-cry/base`: две реплики stateless-приложения.
- `02_ЧАСТЬ_2_SCHEDULER/15_2.2_taint_toleration`: два варианта PostgreSQL.
- На доске: node1 — control plane; node2/node3 — workers.

ПЕРЕХОД

«Начну не с YAML из интернета, а со схемы, которую знает сам API Server».

Время: 04:00–11:00

ЭКРАН: терминал, затем `kubectl explain` рядом с доской Scheduler.

ГОВОРЮ

«`required` — обязательное условие. Если подходящей ноды нет, Pod остаётся Pending. `preferred` — вес в пользу подходящей ноды; Scheduler может выбрать другую, если идеального варианта нет.

Хвост `IgnoredDuringExecution` относится к уже запущенному Pod. Условие проверяется при планировании. Если потом label ноды изменился, kubelet не выселяет Pod только из-за этого. Через несколько минут это проверим на PostgreSQL».

V2-C2-S01-C01 · Зафиксировать self-managed context и спросить API о схеме affinity

```
REPO_ROOT="$(git rev-parse --show-toplevel)"
cd "$REPO_ROOT"
set -a
source recording/.recording.env
set +a
CTX="${SELF_CONTEXT:-kubespray}"
kubectl --context "$CTX" cluster-info
kubectl --context "$CTX" explain pod.spec.affinity
kubectl --context "$CTX" explain pod.spec.affinity.nodeAffinity
kubectl --context "$CTX" explain pod.spec.affinity.podAntiAffinity
```

✅ После команды

Ожидая: отвечает self-managed API; у nodeAffinity и podAntiAffinity видны `requiredDuringSchedulingIgnoredDuringExecution` и `preferredDuringSchedulingIgnoredDuringExecution`.

🧠 ЧТО ЭТО ЗНАЧИТ

`kubectl explain` берёт OpenAPI-схему из текущего API Server. Это надёжнее памяти и случайного блога: поля соответствуют версии нашего кластера.

ПЕРЕХОД

«Теперь зададим нодe две независимые характеристики: label говорит, что это database-нода; taint запрещает случайным Pod'ам на неё заходить».

c2.2 Taint отталкивает, toleration разрешает вход

Время: 11:00–26:00

ЭКРАН: `postgres-pending.yaml` → `postgres-scheduled.yaml` → терминал.

ГОВОРЮ

«На node3 ставлю label `tier=database` и taint `dedicated=database:NoSchedule`. Label сам никого не притягивает. Taint сам не выбирает нужный workload — он только не пускает Pod без пропуска.

Секрет создаю из терминала с выключенным echo. В Git пароль не хранится, на экране его тоже нет. Для настоящего production здесь был бы внешний secret manager или оператор, но механизм Scheduler от этого не меняется».

V2-C2-S02-C01 · Подготовить выделенную database-ноду и секрет без вывода пароля

```
LAB="$REPO_ROOT/02_ЧАСТЬ_2_SCHEDULER/15_2.2_taint_toleration"
kubectl --context "$CTX" create namespace traffic-lab --dry-run=client -o yaml |
kubectl --context "$CTX" apply -f -
kubectl --context "$CTX" describe node node1 | sed -n '/Taints:/p'
kubectl --context "$CTX" label node node3 workload.boostmentor.dev/tier=database
--overwrite
kubectl --context "$CTX" taint node node3 dedicated=database:NoSchedule
read -r -s -p 'temporary DB password: ' DB_PASSWORD; echo
kubectl --context "$CTX" -n traffic-lab create secret generic devops-may-cry-db \
  --from-literal=POSTGRES_DB=devopsmaycry \
  --from-literal=POSTGRES_USER=devopsmaycry \
  --from-literal=POSTGRES_PASSWORD="$DB_PASSWORD" \
  --dry-run=client -o yaml | kubectl --context "$CTX" apply -f -
unset DB_PASSWORD
```

✓ После команды

Ожидаю: node1 уже защищён control-plane taint; node3 получает label и `dedicated=database:NoSchedule`; Secret создан, пароль в терминал не выведен.

🖥️ ПОКАЗАТЬ

Открыть `postgres-pending.yaml` и назвать четыре вещи:

- required nodeAffinity ищет label database;
- toleration отсутствует;
- образ PostgreSQL закреплён digest'ом;
- данные лежат в `emptyDir` — это одноразовая scheduler-лаба, не production storage.

🗣️ ГОВОРЮ

«Этот манифест намеренно неполный. Он хочет node3 по affinity, но не умеет пройти её taint. Применяю и не гадаю по статусу — читаю Events».

📋 V2-C2-S02-C02 · Получить Pending без toleration и прочитать Events

```
kubectl --context "$CTX" apply -f "$LAB/postgres-pending.yaml"
kubectl --context "$CTX" -n traffic-lab get pod -l app=devops-may-cry-postgres -o
wide
kubectl --context "$CTX" -n traffic-lab describe pod -l app=devops-may-cry-
postgres | sed -n '/Events:$/, $p'
```

✓ После команды

Ожидаю: PostgreSQL остаётся `Pending`; Events одновременно показывают требование nodeAffinity и отсутствие toleration для taint database-ноды.

ГОВОРЮ

«В исправленном файле появляется toleration с тем же key, value и effect. Теперь два фильтра согласованы: affinity выбирает класс ноды, toleration разрешает вход.

ConfigMap передаёт SQL-инициализацию отдельно от образа».

ПОКАЗАТЬ

- `postgres-scheduled.yaml`: tolerations и nodeAffinity рядом.
- probes, requests/limits, non-root security context.
- `emptyDir` проговорить как ограничение этой лаборатории.

V2-C2-S02-C03 · Добавить toleration и запустить реальный PostgreSQL на node3

```
kubectl --context "$CTX" -n traffic-lab create configmap devops-may-cry-db-init \
  --from-file=001_init.sql="$LAB/001_init.sql" \
  --dry-run=client -o yaml | kubectl --context "$CTX" apply -f -
kubectl --context "$CTX" apply -f "$LAB/postgres-scheduled.yaml"
kubectl --context "$CTX" -n traffic-lab rollout status deploy/devops-may-cry-
postgres --timeout=120s
kubectl --context "$CTX" -n traffic-lab get pod -l app=devops-may-cry-postgres -o
wide
```

После команды

Ожидая: Pod `Running` именно на node3. Это настоящий PostgreSQL, но данные живут в `emptyDir`: для этой scheduler-лабы они намеренно одноразовые.

В ПРОДЕ

Боевой PostgreSQL обычно живёт в managed-сервисе или под оператором со StatefulSet, CSI/PVC, backup и проверенным restore. Здесь нужен живой процесс базы для Scheduler, поэтому storage одноразовый и это подписано прямо в манифесте.

ПЕРЕХОД

«Pod запущен на node3. Теперь удалю label и проверю, что означает слово Ignored в длинном имени правила».

Время: 26:00–34:00

ЭКРАН: Pod на node3 → удаление label → тот же Pod → пересоздание.

ГОВОРЮ

«Сначала фиксирую имя Pod и ноду. Удаляю label. Запущенный процесс остаётся на месте: Scheduler не пересматривает его каждую секунду, а kubelet не выселяет его из-за nodeAffinity».

V2-C2-S03-C01 · Показать смысл IgnoredDuringExecution

```
kubectl --context "$CTX" -n traffic-lab get pod -l app=devops-may-cry-postgres -o wide
kubectl --context "$CTX" label node node3 workload.boostmentor.dev/tier-
kubectl --context "$CTX" -n traffic-lab get pod -l app=devops-may-cry-postgres -o wide
```

После команды

Ожидаю: уже запущенный PostgreSQL остаётся на node3, хотя label больше не соответствует required nodeAffinity.

ГОВОРЮ

«Удаляю Pod. Deployment создаёт новый, и вот для нового планирования label уже обязателен. Получаем Pending. Возвращаю label — условие снова выполнимо, Pod запускается».

📖 V2-C2-S03-C02 · Пересоздать Pod и вернуть label после доказанного Pending

```
OLD_POD="$(kubectl --context "$CTX" -n traffic-lab get pod \
  -l app=devops-may-cry-postgres -o jsonpath='{.items[0].metadata.name}')"
OLD_UID="$(kubectl --context "$CTX" -n traffic-lab get pod "$OLD_POD" \
  -o jsonpath='{.metadata.uid}')"
kubectl --context "$CTX" -n traffic-lab delete pod "$OLD_POD" --wait=true

NEW_POD=""
for _ in $(seq 1 60); do
  NEW_POD="$(kubectl --context "$CTX" -n traffic-lab get pod \
    -l app=devops-may-cry-postgres \
    -o jsonpath='{range .items[*]}{.metadata.uid}{ " "}{.metadata.name}{ " "}{.status.phase}{ "\n"}{end}' \
    | awk -v old="$OLD_UID" '$1 != old && $3 == "Pending" {print $2; exit}')"
  [ -n "$NEW_POD" ] && break
  sleep 1
done
test -n "$NEW_POD"
kubectl --context "$CTX" -n traffic-lab get pod "$NEW_POD" -o wide
kubectl --context "$CTX" -n traffic-lab describe pod "$NEW_POD" | sed -n
'/Events:/, $p'
kubectl --context "$CTX" label node node3 workload.boostmentor.dev/tier=database
--overwrite
kubectl --context "$CTX" -n traffic-lab rollout status deploy/devops-may-cry-
postgres --timeout=120s
kubectl --context "$CTX" -n traffic-lab get pod "$NEW_POD" -o wide
```

✅ После команды

Ожидаю: цикл находит именно новый UID в фазе `Pending`; Events показывают невозможную nodeAffinity. После возврата label тот же новый Pod становится `Running`.

ЧТО ЭТО ЗНАЧИТ

Affinity — входное условие Scheduler. Это не policy engine, который непрерывно двигает уже запущенные Pod'ы. Для постоянного контроля конфигурации нужны отдельные проверки и автоматика.

ПЕРЕХОД

«С базой закончили. Возвращаю node3 в общий worker-pool и перехожу к доступности stateless-приложения».

c2.4 Pod anti-affinity: полезное правило может заблокировать scale

Время: 34:00–48:00

ЭКРАН: base Deployment → required anti-affinity patch → Pod'ы на двух workers.

ГОВОРЮ

«Удаляю одноразовую базу, secret и configmap, снимаю taint и label. Это обязательный cleanup: в следующей части обе worker-ноды снова нужны приложению.

В base у DEVOPS MAY CRY мягкая anti-affinity и мягкий topology spread. Две реплики стараются разойтись, но правило не блокирует rollout».

V2-C2-S04-C01 · Вернуть workers в общий пул и разложить DEVOPS MAY CRY мягко

```
kubectl --context "$CTX" -n traffic-lab delete deploy/devops-may-cry-postgres
svc/devops-may-cry-postgres --ignore-not-found
kubectl --context "$CTX" -n traffic-lab delete secret/devops-may-cry-db
configmap/devops-may-cry-db-init --ignore-not-found
kubectl --context "$CTX" taint node node3 dedicated=database:NoSchedule-
2>/dev/null || true
kubectl --context "$CTX" label node node3 workload.boostmentor.dev/tier-
2>/dev/null || true
kubectl --context "$CTX" apply -k "$REPO_ROOT/kubernetes/devops-may-cry/base"
kubectl --context "$CTX" -n traffic-lab rollout status deploy/devops-may-cry --
timeout=180s
kubectl --context "$CTX" -n traffic-lab get pod -l app=devops-may-cry -o wide
```

После команды

Ожидая: database-демо удалено, taint/label сняты, две реплики приложения Running и стараются разойтись по node2/node3.

ПОКАЗАТЬ

- `kubernetes/devops-may-cry/base/deployment.yaml`.
- `podAntiAffinity.preferred...`.
- `topologySpreadConstraints`, `topologyKey: kubernetes.io/hostname`.

ГОВОРЮ

«Если добавить required anti-affinity поверх живого RollingUpdate с `maxUnavailable: 0`, surge Pod сам может заблокировать rollout раньше нашей проверки. Поэтому для этой лаборатории делаю воспроизводимый reset: временно scale в ноль, жду удаления старых Pod'ов, применяю required-правило и сразу запускаю три новые реплики. Это лабораторный приём, а не рекомендация останавливать production. Workers только два. Первые две реплики размещаются, третьей физически негде выполнить правило».

📖 V2-C2-S04-C02 · Сделать anti-affinity жёсткой и увидеть цену третьей реплики

```
PATCH="$REPO_ROOT/02_ЧАСТЬ_2_SCHEDULER/17_2.4_pod_anti_affinity/required-anti-affinity-patch.yaml"
kubectl --context "$CTX" -n traffic-lab scale deploy devops-may-cry --replicas=0
POD_COUNT=1
for _attempt in $(seq 1 30); do
    POD_COUNT="$(kubectl --context "$CTX" -n traffic-lab get pod \
        -l app=devops-may-cry --no-headers 2>/dev/null | wc -l | tr -d ' ')"
    test "$POD_COUNT" -eq 0 && break
    sleep 2
done
test "$POD_COUNT" -eq 0
kubectl --context "$CTX" -n traffic-lab patch deploy devops-may-cry --type merge
--patch-file "$PATCH"
kubectl --context "$CTX" -n traffic-lab scale deploy devops-may-cry --replicas=3
RUNNING=0
PENDING=0
for _attempt in $(seq 1 30); do
    RUNNING="$(kubectl --context "$CTX" -n traffic-lab get pod \
        -l app=devops-may-cry --field-selector=status.phase=Running \
        --no-headers 2>/dev/null | wc -l | tr -d ' ')"
    PENDING="$(kubectl --context "$CTX" -n traffic-lab get pod \
        -l app=devops-may-cry --field-selector=status.phase=Pending \
        --no-headers 2>/dev/null | wc -l | tr -d ' ')"
    printf 'running=%s pending=%s\n' "$RUNNING" "$PENDING"
    test "$RUNNING" -eq 2 && test "$PENDING" -eq 1 && break
    sleep 2
done
test "$RUNNING" -eq 2
test "$PENDING" -eq 1
kubectl --context "$CTX" -n traffic-lab get pod -l app=devops-may-cry -o wide
```

```
kubectl --context "$CTX" -n traffic-lab describe pod -l app=devops-may-cry | sed -n '/Events:$/, $p'
```

✅ После команды

Ожидаю: после контролируемого lab-reset на двух workers работают две новые реплики, третья `Pending`; Events объясняют конфликт required podAntiAffinity.

🗣 ГОВОРЮ

«Это нормальный компромисс: required защищает от совместного падения, но может остановить масштабирование и rollout. Для небольшого кластера часто разумнее `preferred`, плюс PDB и запас capacity. Удаляю жёсткое поле и возвращаю две реплики».

📄 V2-C2-S04-C03 · Удалить жёсткое правило и вернуть базовые две реплики

```
kubectl --context "$CTX" -n traffic-lab patch deploy devops-may-cry --type=json \
-
p='[{"op":"remove","path":"/spec/template/spec/affinity/podAntiAffinity/requiredDuringSchedulingIgnoredDuringExecution"}]'
kubectl --context "$CTX" -n traffic-lab scale deploy devops-may-cry --replicas=2
kubectl --context "$CTX" -n traffic-lab rollout status deploy/devops-may-cry --
timeout=180s
```

✅ После команды

Ожидаю: Deployment снова имеет две Ready-реплики; required anti-affinity удалена.

ПЕРЕХОД

«Anti-affinity отвечает на вопрос “не клади рядом с такими Pod'ами”. Topology spread формулирует цель иначе: держи измеримый перекося не больше заданного».

C2.5 Topology spread: maxSkew считаем по фактическим нодам

Время: 48:00–58:00

ЭКРАН: `hard-spread-patch.yaml` → таблица Pod/NODE.

ГОВОРЮ

«Патч задаёт `topologyKey: kubernetes.io/hostname`, `maxSkew: 1` и `DoNotSchedule`. На четырёх репликах ожидаю два Pod'a на node2 и два на node3. На пяти — три и два. Не важны конкретные имена Pod'ов; важна разница между доменами».

V2-C2-S05-C01 · Измерить maxSkew на четырёх и пяти репликах

```
PATCH="$REPO_ROOT/02_ЧАСТЬ_2_SCHEDULER/18_2.5_topology_spread/hard-spread-patch.yaml"
kubectl --context "$CTX" -n traffic-lab patch deploy devops-may-cry --type merge --patch-file "$PATCH"
kubectl --context "$CTX" -n traffic-lab scale deploy devops-may-cry --replicas=4
kubectl --context "$CTX" -n traffic-lab rollout status deploy/devops-may-cry --timeout=180s
kubectl --context "$CTX" -n traffic-lab get pod -l app=devops-may-cry \
  -o custom-columns=NAME:.metadata.name,NODE:.spec.nodeName --sort-by=.spec.nodeName
kubectl --context "$CTX" -n traffic-lab scale deploy devops-may-cry --replicas=5
kubectl --context "$CTX" -n traffic-lab rollout status deploy/devops-may-cry --timeout=180s
kubectl --context "$CTX" -n traffic-lab get pod -l app=devops-may-cry \
  -o custom-columns=NAME:.metadata.name,NODE:.spec.nodeName --sort-by=.spec.nodeName
```

✅ После команды

Ожидаю: четыре Pod'a распределены `2+2`, пять — `3+2`; разница по hostname не больше `maxSkew: 1`.

🧠 ЧТО ЭТО ЗНАЧИТ

`maxSkew: 1` не означает "всегда поровну". Для нечётного числа реплик разница в одну допустима. При недоступной зоне `DoNotSchedule` может оставить Pod Pending; `ScheduleAnyway` разрешает работу ценой временного перекося.

☰ V2-C2-S05-C02 · Вернуть мягкий spread и две реплики

```
kubectl --context "$CTX" -n traffic-lab patch deploy devops-may-cry --type=json \
-
p='[{"op":"replace","path":"/spec/template/spec/topologySpreadConstraints/0/whenUnsatisfiable","value":"ScheduleAnyway"}]'
```

```
kubectl --context "$CTX" -n traffic-lab scale deploy devops-may-cry --replicas=2
```

```
kubectl --context "$CTX" -n traffic-lab rollout status deploy/devops-may-cry --
timeout=180s
```

✅ После команды

Ожидаю: приложение снова Ready с двумя репликами; spread стал предпочтением, а не причиной Pending.

ПЕРЕХОД

«До сих пор ограничения жили в PodСpec. Теперь поставим рамки на весь namespace команды».

c2.6 LimitRange и ResourceQuota работают в паре

Время: 58:00–68:00

ЭКРАН: `limitrange_resourcequota.yaml` → Pod resources → quota Used/Hard.

ГОВОРЮ

«LimitRange задаёт минимум, максимум и defaults для одного контейнера.

ResourceQuota считает суммарное потребление namespace. Это разные задачи.

Создаю отдельный `team-dev`, чтобы учебная квота не сломала `traffic-lab`.

Запускаю nginx без resources. Admission подставляет defaults из LimitRange до того, как quota посчитает запрос».

V2-C2-S06-C01 · Показать, как LimitRange подставляет ресурсы

```
LAB="$REPO_ROOT/02_ЧАСТЬ_2_SCHEDULER/19_2.6_limitrange_quota"
kubectl --context "$CTX" create namespace team-dev
kubectl --context "$CTX" -n team-dev apply -f
"$LAB/limitrange_resourcequota.yaml"
kubectl --context "$CTX" -n team-dev run naked --image=nginx:1.27
kubectl --context "$CTX" -n team-dev get pod naked -o
jsonpath='{.spec.containers[0].resources}'; echo
kubectl --context "$CTX" -n team-dev describe quota
```

После команды

Ожидая: у Pod без явных resources появились default requests/limits; ResourceQuota уже считает их в `Used`.

ПОКАЗАТЬ

- `defaultRequest` и `default`.
- суммарные `requests.cpu`, `limits.cpu`, число Pod/Service/PVC.

- лимит на `services.loadbalancers`: в облаке такой Service может стоить денег.

ГОВОРЮ

«Второй Pod просит 64 CPU. API отклоняет объект до запуска контейнера. Поэтому искать его application logs бессмысленно: процесса не было. После доказательства удаляю весь namespace».

V2-C2-S06-C02 · Получить отказ API на запрос выше квоты и удалить namespace

```
kubectl --context "$CTX" -n team-dev run fat --image=nginx:1.27 \
  --overrides='{"spec":{"containers":
[{"name":"fat","image":"nginx:1.27","resources":{"requests":{"cpu":"64"}}}]}}' ||
true
kubectl --context "$CTX" -n team-dev get events --sort-by=.lastTimestamp | tail
-12
kubectl --context "$CTX" delete namespace team-dev
```

После команды

Ожидаю: API отклоняет Pod как превышающий quota/LimitRange; namespace удалён вместе с учебными объектами.

ПЕРЕХОД

«Финальная привычка этой части: Pending — это статус, а не диагноз».

c2.7 Pending triage: три симптома, три причины

Время: 68:00–79:00

ЭКРАН: три Pod'a → Events каждого → cleanup.

ГОВОРЮ

«Создам три Pending одновременно. Первый требует несуществующий label `disktype=nvme`. Второй просит 64 CPU и 200 GiB памяти. Для третьего временно taint'ю оба workers, а toleration ему не даю.

`kubectl get pods` покажет один и тот же статус. `describe` в Events разделит причины. Это быстрее, чем наугад править image, CNI или приложение».

V2-C2-S07-C01 · Создать три разных Pending и получить три разных диагноза

```
kubectl --context "$CTX" create namespace pending-lab --dry-run=client -o yaml |
kubectl --context "$CTX" apply -f -
kubectl --context "$CTX" -n pending-lab run stuck-selector --image=nginx:1.27 \
  --overrides='{ "spec": { "nodeSelector": { "disktype": "nvme" } } }'
kubectl --context "$CTX" -n pending-lab run stuck-resources --image=nginx:1.27 \
  --overrides='{ "spec": { "containers": [ { "name": "stuck-
resources", "image": "nginx:1.27", "resources": { "requests":
{ "cpu": "64", "memory": "200Gi" } } } ] } }'
kubectl --context "$CTX" taint node node2 maintenance=true:NoSchedule
kubectl --context "$CTX" taint node node3 maintenance=true:NoSchedule
kubectl --context "$CTX" -n pending-lab run stuck-taint --image=nginx:1.27
kubectl --context "$CTX" -n pending-lab get pods
for pod in stuck-selector stuck-resources stuck-taint; do
  echo "=== $pod"
  kubectl --context "$CTX" -n pending-lab describe pod "$pod" | sed -n
  '/Events: /,$p'
done
```

✅ После команды

Ожидая: все три Pod'a `Pending`, но Events различаются: node selector, недостаток CPU/RAM и untolerated taint.

🧠 ЧТО ЭТО ЗНАЧИТ

Порядок triage короткий: Scheduler Events → node labels/taints → allocatable и requests → только потом более редкие причины. Если Pod ещё не назначен на ноду, пустые application logs ожидаемы.

🗣️ ГОВОРЮ

«Перед переходом снимаю оба maintenance-taint и удаляю namespace. Затем проверяю, что DEVOPS MAY CRY снова Ready. Без этого следующая часть началась бы на испорченном стенде».

📋 V2-C2-S07-C02 · Обязательно вернуть ноды и удалить triage-namespace

```
kubectl --context "$CTX" taint node node2 maintenance=true:NoSchedule-
kubectl --context "$CTX" taint node node3 maintenance=true:NoSchedule-
kubectl --context "$CTX" delete namespace pending-lab
kubectl --context "$CTX" -n traffic-lab get deploy,po -l app=devops-may-cry -o
wide
```

✅ После команды

Ожидая: временные taints сняты, pending-lab удалён, DEVOPS MAY CRY остаётся Ready для части 3.

ПЕРЕХОД

«Scheduler выбрал ноду по requests и правилам размещения. В части 3 посмотрим, что после запуска делают limits, cgroups, OOM, throttling, HPA, VPA и Cluster Autoscaler».